

PROMPT ENGINEERING · LLM TECHNIQUES

Advanced Prompting Techniques

A practical guide to the five prompting strategies that most improve LLM output quality — zero-shot, few-shot, chain-of-thought, auto chain-of-thought, and persona prompting — and when to reach for each one.

5

TECHNIQUES

6

PROMPT EXAMPLES

1

DECISION FLOW

10

REVISION CARDS

02 Topic Overview

Five prompting strategies, from simplest to most structured.



Zero-Shot

Ask directly with no examples — relies purely on the model's pretrained knowledge.



Few-Shot

Show 2–5 examples in the prompt so the model can pattern-match the desired output.



Chain-of-Thought

Prompt the model to reason step by step before giving a final answer.



Auto-CoT

Automatically generates diverse reasoning chains instead of hand-writing them.



Persona Prompting

Assigns the model a role or identity to shape tone, style, and expertise.

03 Major Concepts

A detailed breakdown of each prompting technique.

01 Zero-Shot Prompting

BASELINE

DEFINITION

Zero-shot prompting means asking a model to perform a task by simply describing it in natural language — with **no examples** of input/output pairs included. The model relies entirely on knowledge and patterns learned during pretraining and fine-tuning to infer what's being asked.

WHY IT MATTERS

It's the fastest, cheapest way to prompt — no extra tokens spent on examples — and works surprisingly well for common, well-defined tasks (translation, summarization, classification, simple Q&A) because large models have seen similar instructions many times during training.

KEY CONCEPTS

- **Instruction-following** — relies on the model's fine-tuning to interpret plain instructions.
- **No demonstration bias** — output isn't skewed by example formatting choices.
- **Best for common tasks** — degrades on novel, ambiguous, or highly specific formats.

EXAMPLE

```
Classify the sentiment of this review as Positive, Negative, or Neutral:  
"The battery life is disappointing but the camera is excellent."
```

ANALOGY

Zero-shot is like asking a well-read colleague a question cold, with no background — they answer using everything they already know, without you needing to show them how it's done first.

COMMON MISTAKE

Using zero-shot for tasks that need a very specific output format (e.g. exact JSON schema, custom tone, unusual structure). Without examples, the model guesses at formatting and often gets it inconsistently right — few-shot is safer for strict formatting needs.

02 Few-Shot Prompting

PATTERN MATCHING

DEFINITION

Few-shot prompting includes a handful (typically 2–5) of input/output **demonstrations** directly in the prompt before the actual query, letting the model infer the task's pattern, format, and style purely from context — a form of **in-context learning** with no weight updates.

WHY IT MATTERS

It dramatically improves reliability for tasks with a specific output structure, tone, or edge-case handling that's hard to describe in words but easy to demonstrate. It's often the difference between an 80%-correct format and a 99%-correct one, without any fine-tuning.

KEY CONCEPTS

- **In-context learning** — the model adapts behavior purely from prompt content, not training.
- **Example diversity** — varied examples generalize better than near-duplicate ones.
- **Example ordering** — order and recency can bias the model's output; shuffle when testing.
- **Token cost** — each example consumes context-window budget and adds latency/cost.

EXAMPLE

```
Review: "Loved the fast shipping!" → Positive
Review: "Item arrived broken." → Negative
Review: "It's okay, does the job." → Neutral
Review: "The battery life is disappointing but the camera is excellent." → ?
```

🔑 KEY POINT

Few-shot examples act as a lightweight, temporary "specification" for the task — they teach format and style far more reliably than instructions alone, especially for structured or domain-specific output.

⚠️ COMMON MISTAKE

Using examples that are all near-identical in structure or all one label (e.g. all "Positive" reviews). This biases the model toward repeating that pattern rather than truly generalizing the task.

03 Chain-of-Thought (CoT) Prompting

REASONING

DEFINITION

Chain-of-thought prompting instructs the model to generate intermediate **reasoning steps** — explaining its logic out loud — before producing a final answer, rather than jumping straight to a conclusion. This is often triggered with phrasing like "Let's think step by step."

WHY IT MATTERS

Multi-step reasoning tasks — arithmetic, logic puzzles, multi-hop questions, code debugging — are where models fail most when forced to answer immediately. Spelling out intermediate steps gives the model "room to think," using its own generated tokens as working memory, which measurably improves accuracy on complex tasks.

KEY CONCEPTS

- **Zero-shot CoT** — simply appending "Let's think step by step" with no examples.
- **Few-shot CoT** — providing worked examples that include reasoning steps, not just answers.
- **Self-consistency** — sampling multiple CoT paths and taking a majority-vote final answer.
- **Token-as-compute** — reasoning tokens effectively give the model more "thinking time."

EXAMPLE

```
Q: A store had 23 apples. It sold 8 and received a delivery of 15 more. How many apples now?
A: Let's think step by step.
Start: 23 apples. Sold 8 → 23 - 8 = 15. Delivery of 15 → 15 + 15 = 30.
Final answer: 30 apples.
```

💡 ANALOGY

CoT is like asking someone to "show their work" on a math test instead of just writing the final number. Showing work not only lets you check their logic — it often helps them catch their own mistakes along the way, arriving at a more accurate answer.

PROMPT • ZERO-SHOT CoT

```
# System / user prompt
"""
Question: Roger has 5 tennis balls. He buys 2 more cans of
tennis balls. Each can has 3 balls. How many tennis balls
does he have now?
```

Let's think step by step.

```
"""  
  
# Model appends its own reasoning, then a final answer line,  
# e.g. "Final answer: 11"
```

04 Auto Chain-of-Thought (Auto-CoT)

AUTOMATION

DEFINITION

Auto-CoT automates the construction of few-shot chain-of-thought examples instead of a human hand-writing them. It works in two stages: **(1) question clustering** — grouping a set of sample questions by similarity — and **(2) demonstration generation** — using zero-shot CoT ("let's think step by step") to auto-generate a reasoning chain for one representative question per cluster, which are then used as the few-shot examples for new queries.

WHY IT MATTERS

Hand-writing diverse, high-quality CoT examples is slow and doesn't scale across many task types. Auto-CoT removes that manual effort while still getting the accuracy benefits of few-shot CoT, and its clustering step helps avoid the "repeated mistake" problem where hand-picked examples happen to share a reasoning error the model copies.

KEY CONCEPTS

- **Question clustering** — groups similar questions to ensure diverse demonstration coverage.
- **Auto-generated rationales** — reasoning chains are produced by the model itself via zero-shot CoT.
- **Diversity over hand-curation** — reduces the risk of biased or repetitive manual examples.
- **Scales across tasks** — one pipeline can build CoT demonstrations for many different domains.

EXAMPLE

```
Step 1: Cluster 50 sample math questions into 8 groups by embedding similarity.  
Step 2: For one question per cluster, run zero-shot CoT to auto-generate a worked solution.  
Step 3: Use those 8 generated Q→reasoning→answer pairs as the few-shot prompt for new questions.
```

⚡ KEY POINT

Auto-CoT was introduced specifically to address a weakness of manual few-shot CoT: if a hand-written example contains a subtle reasoning error, the model tends to imitate that error on similar new questions. Diverse, auto-generated demonstrations reduce this risk.

⚠ COMMON MISTAKE

Skipping the clustering step and just auto-generating CoT examples from random questions. Without diversity, the auto-generated demonstrations can end up too similar to each other, losing the accuracy benefit that clustering was designed to provide.

05 Persona Prompting

STYLE CONTROL

DEFINITION

Persona prompting assigns the model a specific role, identity, or point of view — e.g. "You are a senior security engineer" — typically in the system prompt, to steer its tone, vocabulary, priorities, and depth of expertise for the rest of the conversation.

WHY IT MATTERS

The same underlying question can warrant very different answers depending on audience and purpose. A persona focuses the model's vast pretrained knowledge toward the relevant subset — a "beginner-friendly teacher" persona produces simpler language than a "PhD-level researcher" persona, without changing the literal question asked.

KEY CONCEPTS

- **System-level persona** — set once, applies to the whole conversation.
- **Domain framing** — role choice implicitly signals which knowledge/vocabulary to prioritize.
- **Tone & audience control** — shapes formality, simplicity, and structure of responses.
- **Combinable** — persona pairs well with few-shot and CoT for a fully-specified prompt.

EXAMPLE

```
System: You are a patient, encouraging math tutor for a 10-year-old student.  
Explain concepts using simple language and everyday analogies.
```

```
User: Why does 1/2 + 1/3 not equal 2/5?
```

💡 ANALOGY

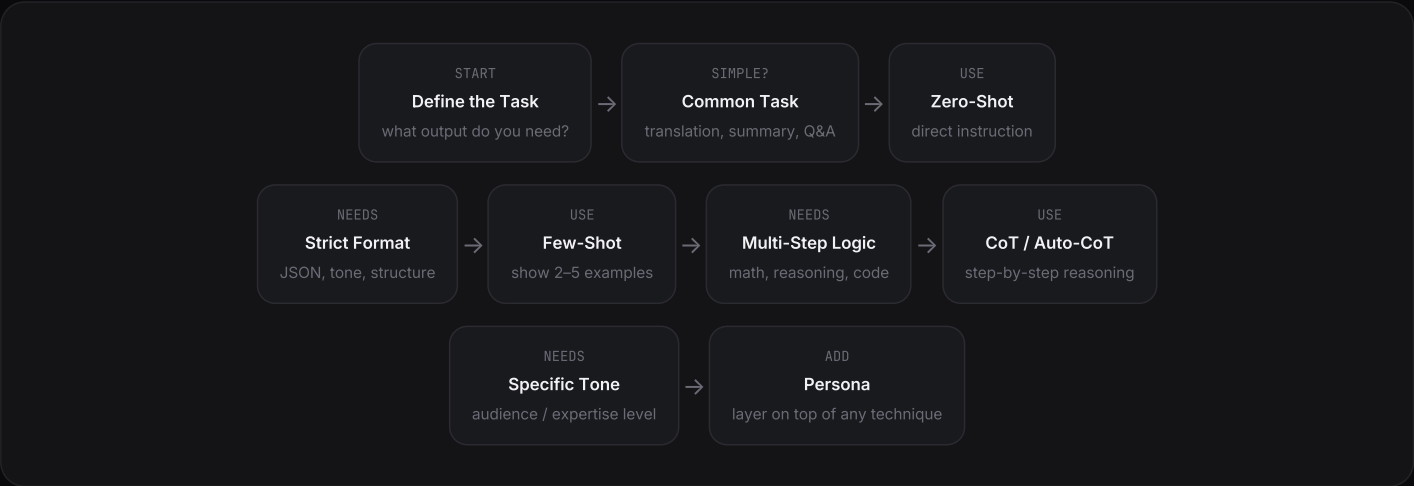
Persona prompting is like briefing an actor before a scene — "play this as a weary detective, not a cheerful host" — the underlying script (the model's knowledge) doesn't change, but the delivery does.

⚠️ COMMON MISTAKE

Treating a persona as a substitute for real constraints. Saying "You are an expert lawyer" doesn't guarantee legally accurate output — it shapes tone and framing, not factual correctness, so critical claims still need verification.

04 **Workflow: Choosing a Technique**

A quick decision path from task to prompting strategy.



05 **Step-by-Step: Building a Combined Prompt**

Layering persona, few-shot examples, and chain-of-thought into one strong prompt.

- 1 Start with a clear task statement**
State exactly what output you need in one or two plain sentences before adding anything else.
- 2 Add a persona if tone/expertise matters**
Set a system-level role like `"You are a senior data analyst"` to anchor vocabulary and depth.
- 3 Add few-shot examples for format-sensitive tasks**
Include 2–5 diverse input → output pairs that demonstrate the exact structure you expect back.
- 4 Add "let's think step by step" for reasoning tasks**
Append a CoT trigger phrase, or bake reasoning steps directly into your few-shot examples.
- 5 Test, then trim**
Remove any example or instruction that doesn't measurably improve output — every extra token costs latency and money.

```
PROMPT · PERSONA + FEW-SHOT + COT COMBINED

# System
You are a meticulous senior data analyst who explains reasoning clearly before giving a final number.

# Few-shot example
Q: Revenue was $120k in Jan and grew 10% each month for 3 months.
   What is the revenue in April?
A: Let's think step by step.
   Jan: 120k. Feb: 120k * 1.10 = 132k. Mar: 132k * 1.10 = 145.2k.
   Apr: 145.2k * 1.10 = 159.72k.
   Final answer: $159.72k

# Actual query
Q: Revenue was $80k in Jan and grew 8% each month for 4 months.
   What is the revenue in May?
A: Let's think step by step.
```

06 **Comparison: When to Use Each Technique**

TECHNIQUE	BEST FOR	TOKEN COST	TRADE-OFF
Zero-Shot	Common, well-defined tasks	Lowest	Less reliable on niche formats
Few-Shot	Strict formatting, style matching	Medium	Examples add tokens and can bias output
Chain-of-Thought	Math, logic, multi-step reasoning	Medium-High	Longer output, higher latency/cost

Auto-CoT	Scaling CoT across many task types	High (setup)	Requires a clustering/generation pipeline
Persona	Tone, audience, expertise framing	Low	Doesn't guarantee factual accuracy

07 Final Summary

What to remember

- ✓ **Zero-shot** is fastest and cheapest — start here for common, well-defined tasks.
- ✓ **Chain-of-thought** improves multi-step reasoning by making the model "show its work."
- ✓ **Persona** shapes tone and expertise level, and layers on top of any other technique.
- ✓ **Few-shot** teaches format and style through 2–5 concrete demonstrations.
- ✓ **Auto-CoT** automates diverse reasoning-example generation via clustering.
- ✓ Techniques **combine** — a strong prompt often mixes persona + few-shot + CoT together.

08 Quick Revision Cards

🎯 Zero-Shot

- No examples given
- Fast, cheap, good for common tasks
- Weaker on strict formatting

📄 Few-Shot

- 2–5 demonstrations in-prompt
- In-context learning, no fine-tuning
- Use diverse, non-repetitive examples

🧠 Chain-of-Thought

- "Let's think step by step"
- Boosts math/logic accuracy
- Self-consistency = majority vote

🔍 Auto-CoT

- Cluster questions → auto-generate reasoning
- Removes manual example-writing
- Diversity avoids repeated errors

👤 Persona

- Assigns a role in the system prompt
- Controls tone, not correctness
- Combines well with other techniques

🧩 Combining Techniques

- Persona + few-shot + CoT layer together
- Trim anything that doesn't help
- Always test on real edge cases

09 Glossary / Terminology Cheat-Sheet

In-Context Learning

The model adapting its behavior based on prompt content alone, with no weight updates.

System Prompt

A standing instruction set before the conversation that shapes behavior throughout.

Rationale

The intermediate reasoning text a model produces on the way to a final answer.

Self-Consistency

Sampling several CoT reasoning paths and choosing the most common final answer.

Demonstration

A single example input/output pair included in a few-shot prompt.

Prompt Injection

Untrusted input manipulating a model's instructions — a key risk to guard against in prompt design.

Trigger Phrase

A short instruction (e.g. "let's think step by step") that reliably invokes a reasoning behavior.

Clustering

Grouping similar questions together, used in Auto-CoT to ensure diverse demonstrations.

Prompt Template

A reusable, parameterized prompt structure applied consistently across many inputs.

Grounding

Anchoring a model's output in supplied facts or context rather than its own memory alone.

Hallucination

Fluent but factually incorrect output — persona and CoT reduce but do not eliminate this risk.